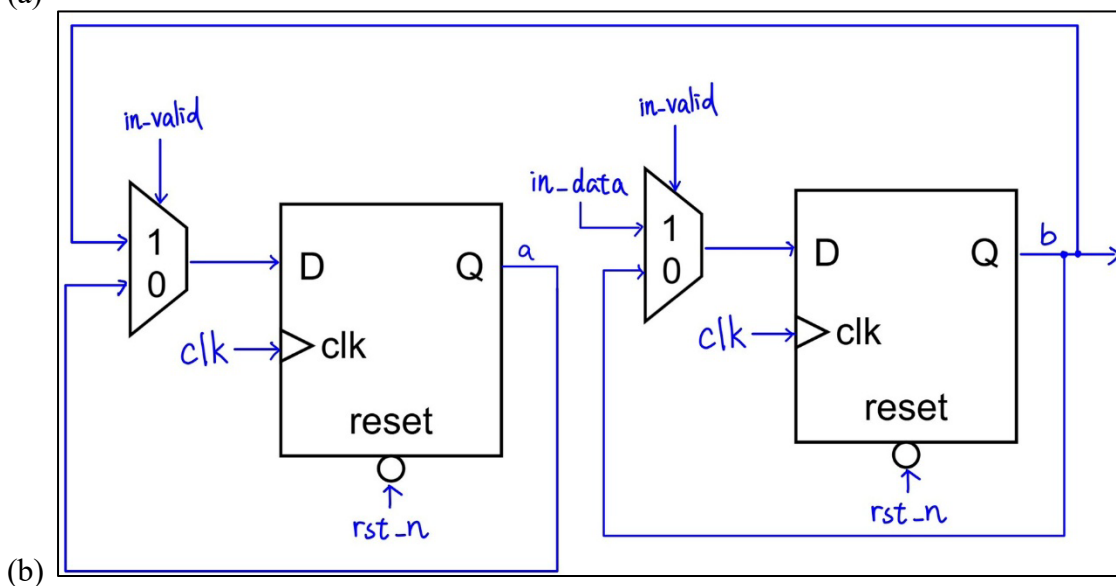
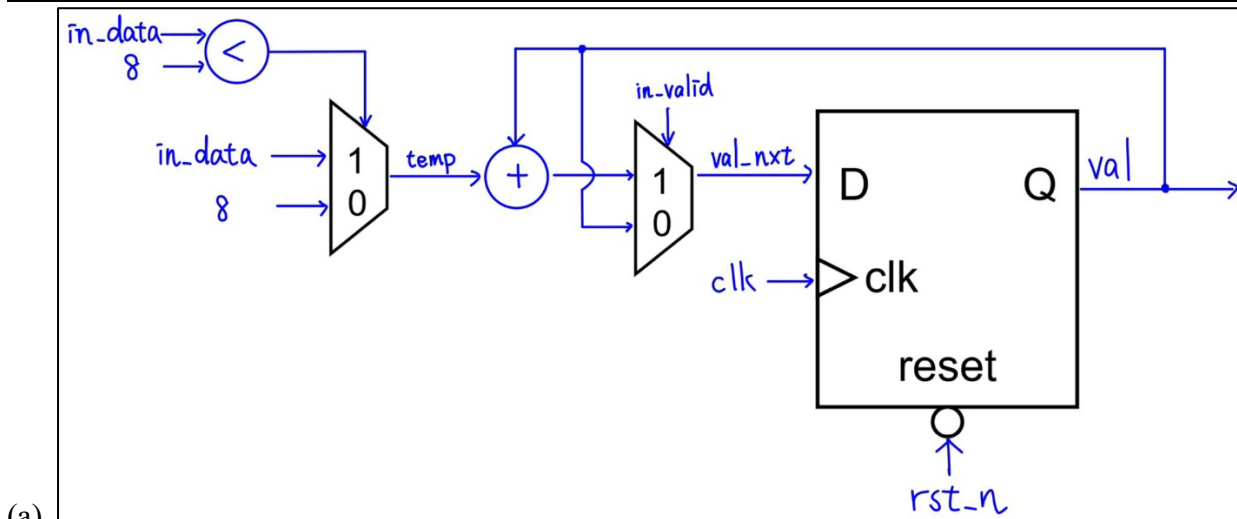


NYCU-ECE DCS-2026

Midterm Answer

1. Block Diagram (14%)

(a) (7%)	(b) (7%)
<pre> always_ff @ (posedge clk or negedge rst_n) begin if (!rst_n) val <= 0; else val <= val_nxt; end always_comb begin temp = (in_data < 8) ? (in_data) : 8; val_nxt = val; if (in_valid) val_nxt = val + temp; end </pre>	<pre> always_ff @ (posedge clk or negedge rst_n) begin if (!rst_n) begin a <= 0; b <= 0; end else begin if (in_valid) begin a <= b; b <= in_data; end end end </pre>



2. Please answer the following question. (29%)

(a) [*Truncation and Extension*] Please determine the complete **binary** representation according to its specified bit-width for the following constants. (e.g., the answer of 3'd2 is 010)

(i) 6'ha (ii) -3'b1 (iii) 4'sd6 (iv) 7'hb9 (v) 7'o71

(i) **001010** (ii) **111** (iii) **0110** (iv) **0111001** (v) **0111001**

(b) [*timescale*] Please complete the following table. (ns = 10^{-9} s, ps = 10^{-12} s) (四捨五入錯誤 1 個-0.5)

Unit / Precision	Delay Spec.	Effective Delay
1ns / 1ps	#5.0093	5.009 ns
10ns / 1ns	#8.7681	88 ns
10ns / 100ps	#6.9281	69.3 ns
1ns / 100ps	#4.2970	4.3 ns

(c) [*SystemVerilog Data Types*] Compare the differences between 4-state and 2-state data types.

1. What are the possible values of each? **4-state: 0,1,X,Z** **2-state: 0,1**

2. What are the **uninitialized nets** values of each? **4-state: Z** **2-state: 0**

3. Provide 2 examples for each.

4-state: logic, integer, reg, wire **2-state: int, shortin, longint, bit, byte, real**

(d) [*task / function*] Answer **T** (true) or **F** (false) to the following statements. If the answer is false, please give a brief reason.

1. Both task and function blocks can contain timing control (@, #, ...). **F**

2. Both task and function blocks can have multiple inputs. **T**

3. Task blocks can have multiple inputs, but function blocks can only have 1 input. **F**

4. Task blocks can have multiple outputs, but function blocks can only have 1 output. **T**

5. Both task and function blocks cannot be synthesized. **F**

(e) [*Timing Report*]

i. Formulate the setup / hold time **inequalities**. (use $T_c, t_{pd}, t_{pcq}, t_{setup}, t_{cd}, t_{ccq}, t_{hold}$)

ii. Determine if there is a setup time violation in Fig. 1. You must calculate the **data arrival / required time** and the **setup slack**. Assume the circuit is operated under clock cycle time = 15 ns.

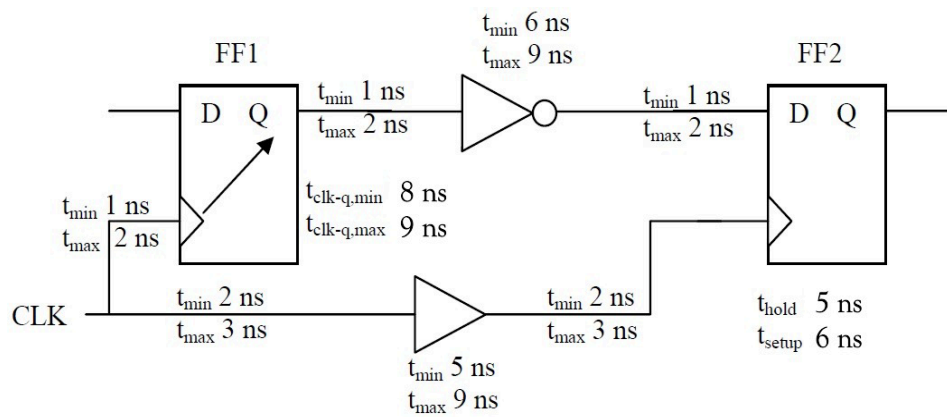
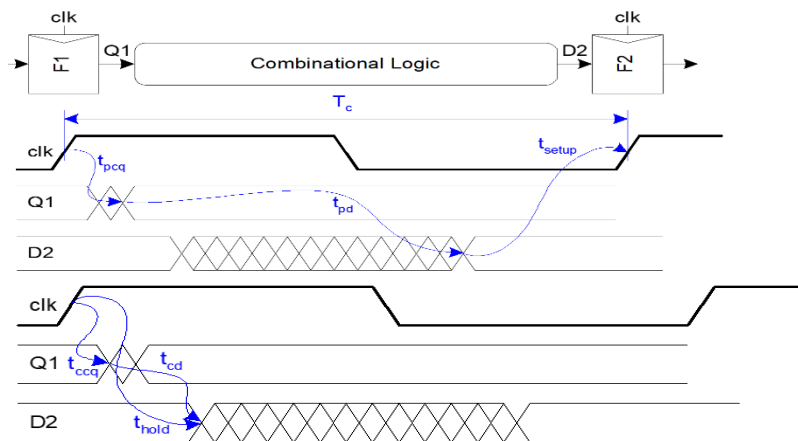


Fig.1



i. $T_c \geq t_{pd} + t_{pcq} + t_{setup}$

$t_{cd} + t_{ccq} \geq t_{hold}$

ii. data arrival time: $2+9+2+9+2=24\text{ns}$

data required time: $15+2+5+2-6=18\text{ns}$

slack: $18-24=-6\text{ns}$ (Setup Violation)

3. (12%)

寫 (posedge clk or negedge rst_n) +2

寫 (*) +2

寫 if(!rst_n) +2

寫到 xor 的概念 +2

寫到 toggle 的概念 +2

(最終波形錯者最多 10 分)

out 沒用 assign -1 (net type)

(最終波形對者最少 10 分)

```

1 module ASIC (
2     input rst_n,
3     input clk,
4     input in1,
5     input in2,
6     output out
7 );
8
9 logic out_cs;
10 logic out_ns;
11
12 assign out = out_cs;
13
14 always @(posedge clk or negedge rst_n) begin //sequential circuit
15     if (!rst_n) begin
16         out_cs <= 0;
17     end else begin
18         out_cs <= out_ns;
19     end
20 end
21
22 always @(*) begin //combinational circuit
23     out_ns = (in1 ^ in2) ? (!out_cs) : (out_cs);
24 end
25
26 endmodule

```

助教依整體情況會酌情增減分數 (<6: 錯很多 6: 基本分 8: 對一半 10: 臨門一腳 11: 沒寫 assign)

4. Debug (25%):

Please find out **6 errors** (that cause RTL simulation failed, synthesis failed, or behavior inconsistency between RTL and gate-level simulation) in the design below. For each error you identified, circle it and briefly explain why it causes an error and how to fix it. Assume that both the input external delay and output external delay are equal to half the clock cycle. Finding more errors will not be penalized.

Note: Errors of the same type can only be counted as one!!!

```
1 module data_processor (
2     input clk, rst_n, enable, mode_sel,
3     input [31:0] data_bus,
4     input [1:0] sel_idx,
5     output logic out_valid, err_flag,
6     output logic [7:0] out_byte, out_status
7 );
8 logic [1:0] state, state_next;
9 logic [7:0] ext_data, shifted_data, parity_val, out_status_next;
10 logic ready_wire, status_wire, out_valid_next, err_flag_next;
11
12 function logic [7:0] check_shamt(input logic [7:0] data);
13     if (!data[0]) begin
14         err_flag_next = 1'b1;
15         return ~data;
16     end
17     err_flag_next = 1'b0;
18     return data;
19 endfunction
20
21 assign err_flag_next = mode_sel & ~enable;
22 assign ready_wire = status_wire & mode_sel;
23 assign parity_val = mode_sel ? (data_bus[7:0] ^ data_bus[15:8]) : 8'h00;
24
25 always_comb begin
26     if (enable)
27         status_wire = ready_wire | parity_val[0];
28     else
29         status_wire = 1'b0;
30
31     ext_data = data_bus[sel_idx*8 + 7 : sel_idx*8];
32     shifted_data = check_shamt(ext_data);
33
34     while (shifted_data[7] == 1'b0) begin
35         shifted_data = shifted_data << 1;
36     end
37 end
38
39 always_comb begin
40     state_next = state;
41     out_valid_next = 1'b0;
42     case (state)
43         2'd0: begin
44             if (enable) state_next = 2'd1;
45             out_status_next = 8'h00;
46         end
47         2'd1: begin
48             if (ready_wire) state_next = 2'd2;
49             out_status_next = 8'h11;
50         end
51         2'd2: begin
52             state_next = 2'd0;
53             out_valid_next = 1'b1;
54         end
55         default: begin
56             state_next = 2'd0;
57             out_status_next = 8'hFF;
58         end
59     endcase
60 end
61
62 always_ff @(posedge clk or negedge rst_n) begin
63     if (!rst_n || !enable) begin
64         state <= 2'd0; out_valid <= 1'b0;
65         out_byte <= 'd0; err_flag <= 1'b0;
66     end else begin
67         state <= state_next; out_valid <= out_valid_next;
68         out_byte <= shifted_data; err_flag <= err_flag_next;
69     end
70 end
71
72 always_ff @(posedge clk) out_status <= out_status_next;
73
74 endmodule
```

ERROR 1: (multi-driver +5)

Line 14 & 17 or 21: Multiple drivers for 'err_flag'.

```
Error-[ICPD] Illegal combination of drivers
test.sv, 8
Illegal combination of procedural drivers
Variable 'err_flag' is driven by an 'always' combination of procedural
drivers. Variables written on left-hand of 'always' cannot be written
to by any other processes, including other 'always' processes.
```

Remove 'err_flag' assignment from either one.

ERROR 2: (comb. loop +5)

Line 22 & 27: Combinational loop (ready_wire <-> status_wire).

```
Information: Checking loops...
Warning: There are 1 timing loops in the design. (TIM-209)
```

Break the loop by assigning 'ready_wire' to an independent signal.

```
always_comb begin
    x = 1;
    x = x + 1;
end
```

ERROR 3: (cannot put variable +5)

Line 31: Non-constant part-select bounds

```
Error-[IRIPS] Illegal range in part select
test.sv, 45
The range of the part select is illegal:
Unknown range in part select.data_bus[((sel_idx * 8) + 7):(sel_idx * 8)]
```

Use indexed part-select: ext_data = data_bus[sel_idx*8 +: 8];

Or use barrel shifter: ext_data = data_bus >> (sel_idx * 8);

ERROR 4: (cannot be synthesized +5; don't use while +3)

Line 34-36: Unbounded loop condition.

```
Error: ../01_RTL/test.sv:65: Loop exceeded maximum iteration limit. (ELAB-900)
```

Use a Priority Encoder (casez) to perform a single shift.

```
casez (shifted_data)
  8'b01??_????: shifted_data = shifted_data << 1;
  8'b001?_????: shifted_data = shifted_data << 2;
  8'b0001_????: shifted_data = shifted_data << 3;
  8'b0000_1???: shifted_data = shifted_data << 4;
  8'b0000_01??: shifted_data = shifted_data << 5;
  8'b0000_001?: shifted_data = shifted_data << 6;
  8'b0000_0001: shifted_data = shifted_data << 7;
endcase
```

ERROR 5: (not mention about latch +3)

Line 51-54: Incomplete condition for 'out_status_next'. Latch inferred.

out_status_next_reg	Latch	8	Y	N	None	None	-	39	
---------------------	-------	---	---	---	------	------	---	----	--

Assign 'out_status_next' in this branch or provide a default value before 'case'.

ERROR 6: (separate +5)

Line 63: Invalid async reset condition

```
Error: ../01_RTL/test.sv:84: The expression in the reset condition of the 'if' statement in this 'always' block can only be a simple identifier or its negation. (ELAB-303)
```

Separate '!rst_n' and '!enable'. Async reset must be the sole condition in the first 'if'.

5. (20%) Please draw the corresponding RTL simulation waveform (01_RTL) according to the following design.

Note: Please pay attention to the timing of the DFF trigger and reset, and clearly illustrate the start and end positions of each value. (Hint: c and d have a short period of "unknown" state.)

a~d 每個訊號 5 分，錯一個 cycle 扣 1 分，5 分扣完為止

